

SHRINAV WORKSHOP

Practice Assignment

API Fundamentals & Postman Mastery

Complete every task in Postman using the JSONPlaceholder API. Read each instruction carefully — later tasks build on what you did in earlier ones.

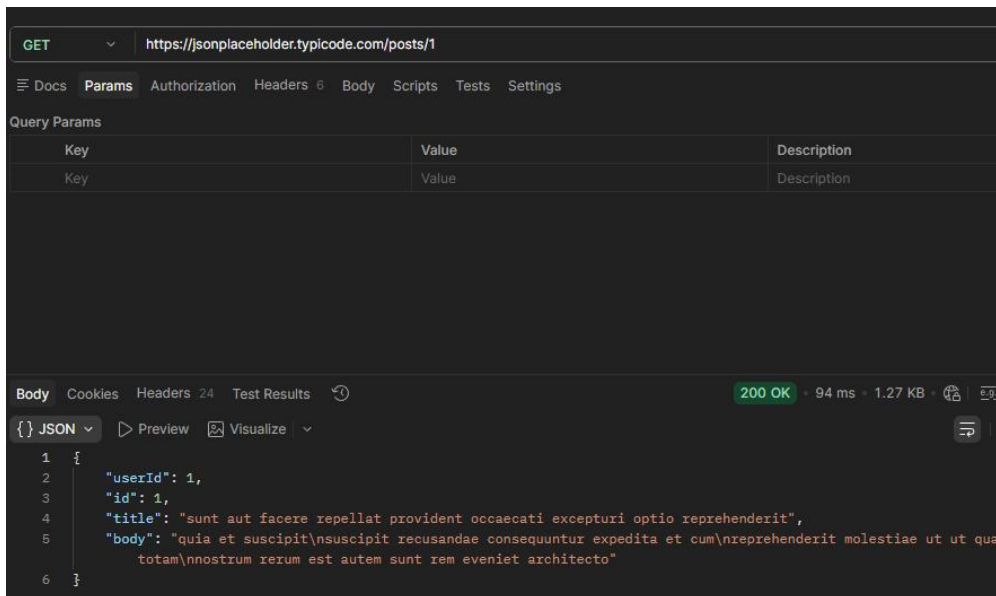
Before you start: you only need a browser and Postman. Base URL for every task: <https://jsonplaceholder.typicode.com> — no signup or API key required.

Tasks

TASK 1 Reading Data Without Headers

- Send a GET request to `/posts/1` — leave the Headers and Body tabs completely empty.
- Record the status code you get back.
- In one sentence, explain why this request works even with nothing in the Headers tab.

A)



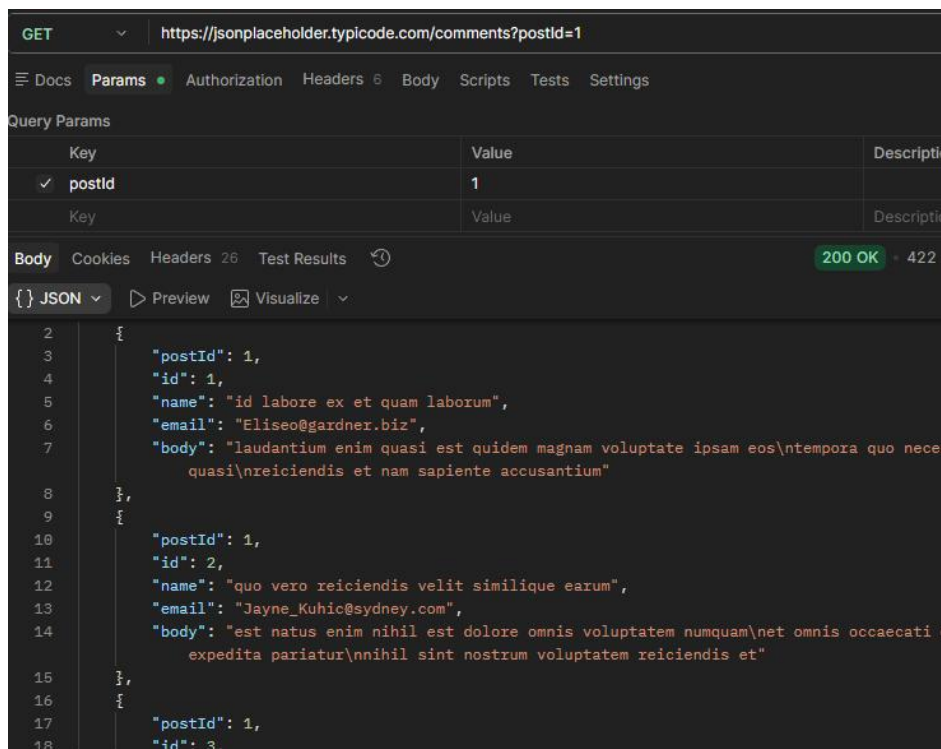
B) 200 OK

C) This request works without headers because it is a simple GET request, and the server can return public data without requiring additional information from the client.

TASK 2 Filtering With Query Parameters

- Send a GET request to /comments, and add a query parameter in the Params tab: postId = 1.
- Confirm the URL automatically updates to include ?postId=1.
- Now combine two filters at once on /todos: userId = 1 and completed = false.
- In one sentence, explain what changed in the results compared to not using any params.

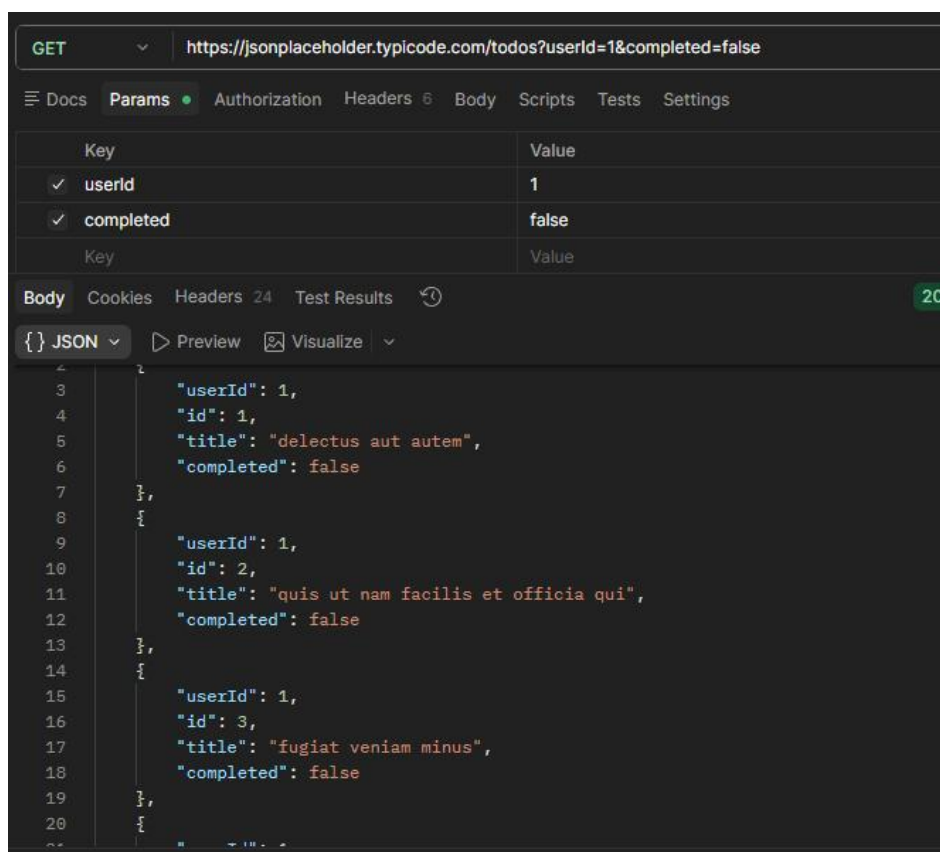
A)



B)yes ,Postman automatically changes the URL to

<https://jsonplaceholder.typicode.com/comments?postId=1>

C)

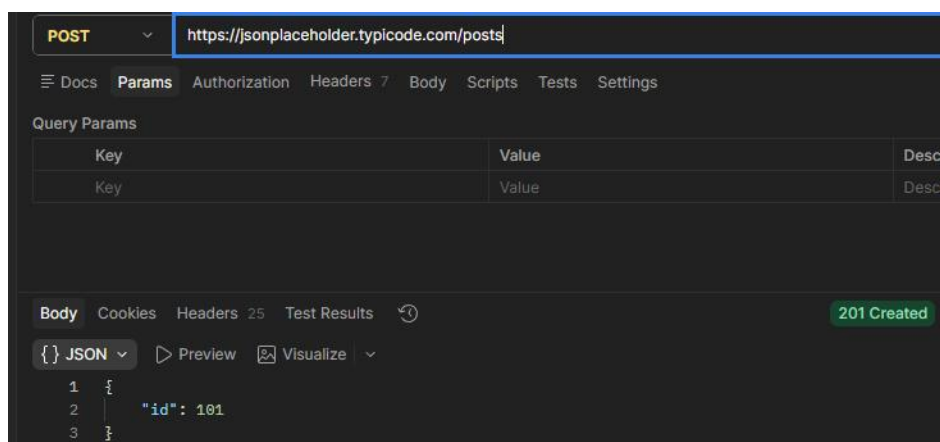


D) Query parameters filter the results, so the API returns only records that match the specified conditions instead of returning all available data.

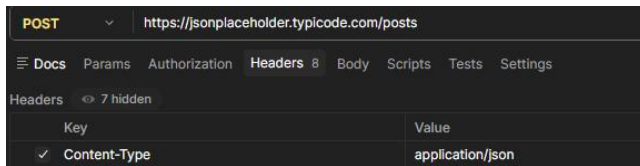
TASK 3 Sending Data With Headers & a Body

- Send a POST request to /posts.
- In the Headers tab, add: Content-Type: application/json
- In the Body tab (raw, JSON), send a new post with a title, body, and userId of your choice.
- Record the status code and the new id the server assigned.

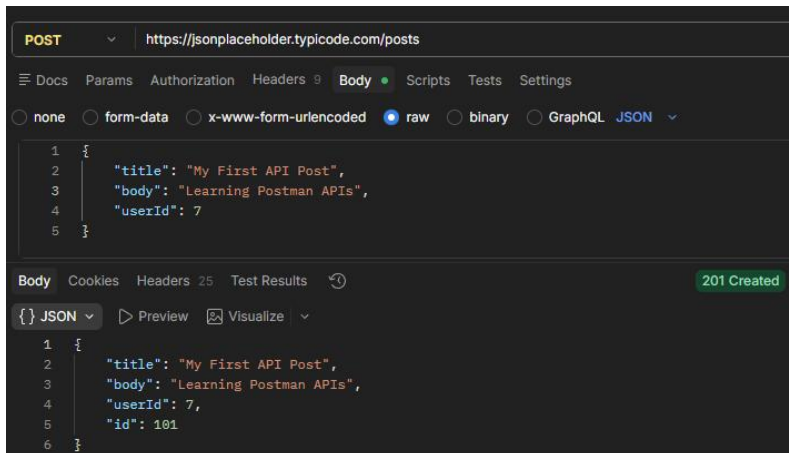
A)



B)



C)



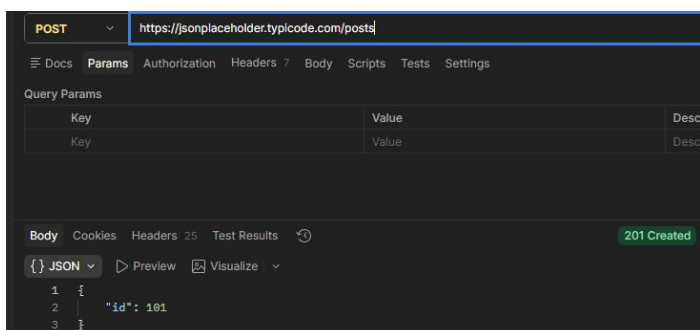
D) Status Code : 201 Created

New ID : 101

TASK 4 Headers vs. No Headers

- Repeat Task 3's POST request, but this time remove the Content-Type header before sending.
- Compare the response to Task 3 — is it different? Record what you observe.
- In 2–3 sentences, explain why headers matter for a request like this, even though this practice API is forgiving about it.

A)



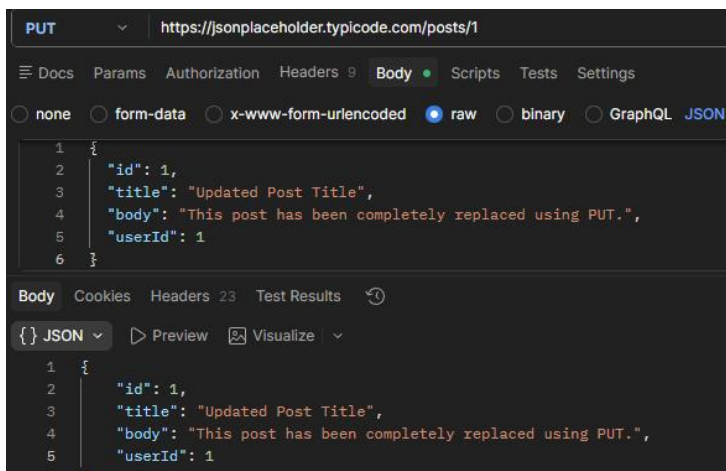
B) The response was almost the same as Task 3. Even without the Content-Type: application/json header, the API returned **201 Created** and generated a new post because JSONPlaceholder accepts the request without strict header validation.

C) In real APIs, the Content-Type header tells the server how to interpret the request body. Without it, many servers reject the request with an error because they cannot determine the data format.

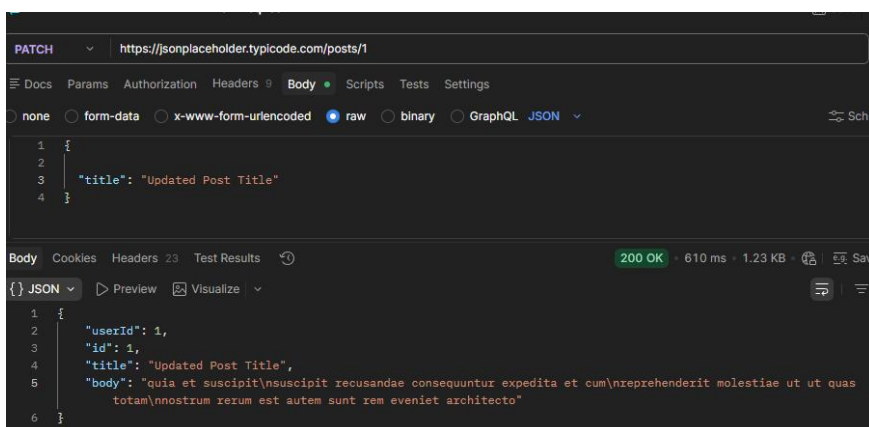
TASK 5 PUT vs. PATCH

- Send a PUT request to `/posts/1` that replaces the entire post (include id, title, body, and userId in the body).
- Send a PATCH request to the same `/posts/1` that changes only the title.
- In one sentence, explain the difference between what PUT sent and what PATCH sent.

A)



B)



C) PUT sends the complete resource and replaces all of its fields, whereas PATCH sends only the fields that need to be updated, leaving the remaining fields unchanged.

Final Challenge — A Realistic Request Flow

Concepts used: this task combines query params, headers (with and without), and the request body — everything from Tasks 1 through 4 — in one flow.

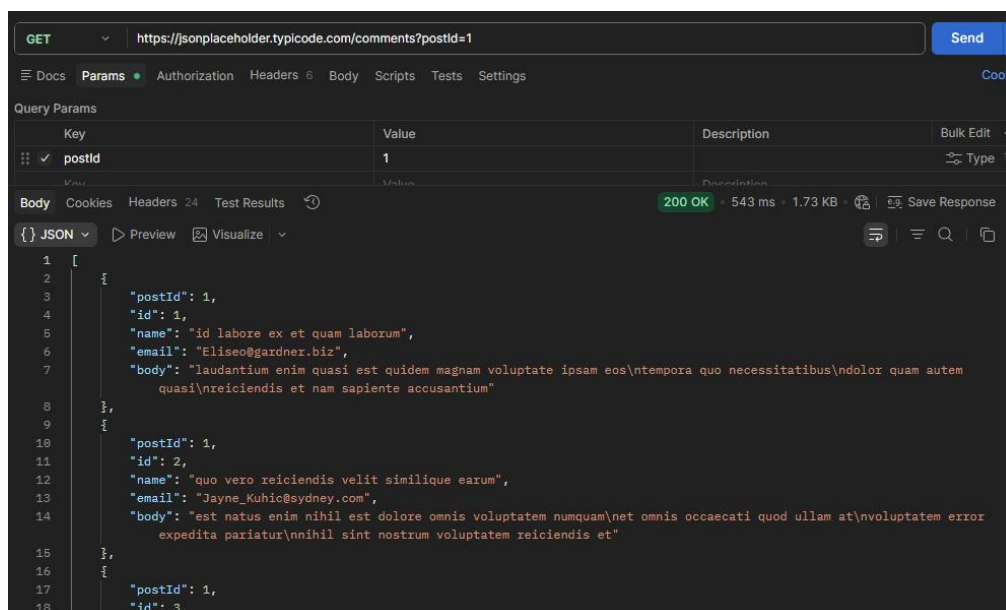
TASK 6 The Mini Feedback System

Scenario: you're testing the backend for a simple feedback form, where users read existing comments on a post and can leave their own.

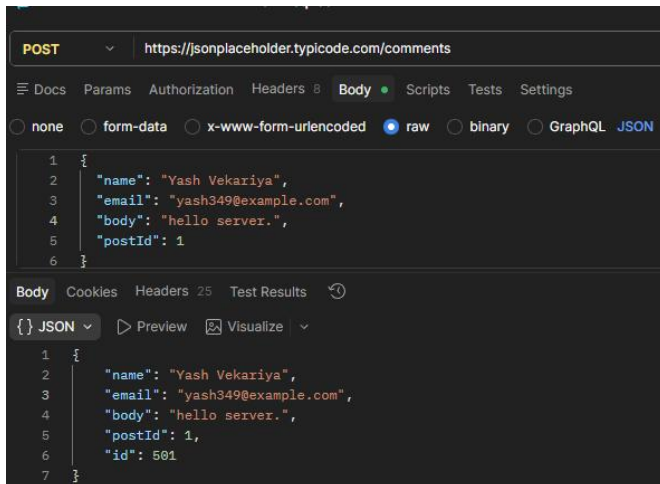
Complete the following steps, in order:

- a) Send a GET request to /comments, using a query parameter to fetch only the comments for postId = 1.
- b) Send a POST request to /comments to add a new comment — with no headers at all. Record the status code.
- c) Resend the same POST from step (b), this time with the Content-Type header included. Compare the two responses.
- d) Add one more header to this request: Authorization: Bearer demo-token — to simulate a login-protected submission, even though this practice API won't actually check it.
- e) Record the status code for every step above (a–d) in a short table or list.
- f) In 2–3 sentences, explain what would happen differently at step (d) if this were a real API that actually required authentication.

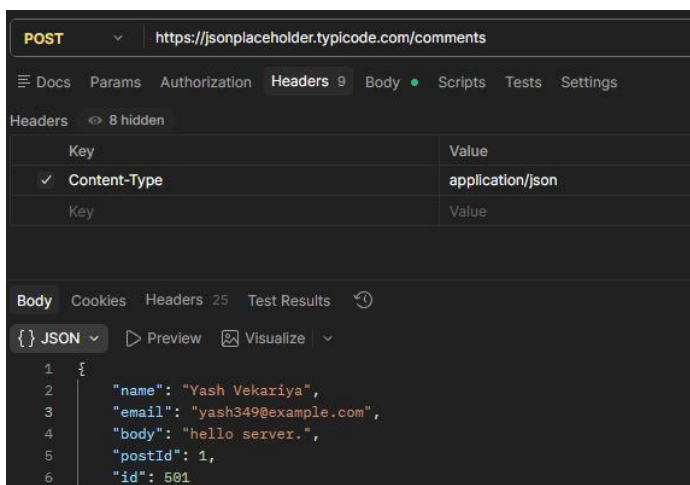
A)



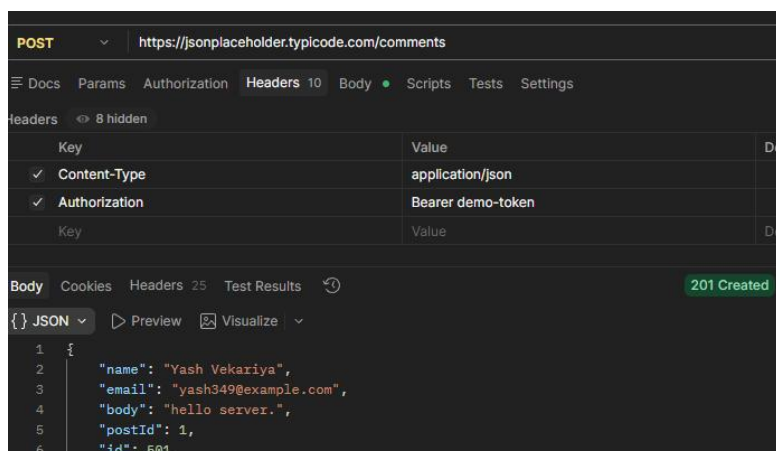
B)



C)



The response is almost identical in both cases. JSONPlaceholder accepts the request even without the Content-Type header because it is a mock API, but in a real API this header is important for correctly interpreting the request body.



D)

E)

Step	Request	Status Code
a	GET /comments?postId=1	200 OK
b	POST /comments (No Headers)	201 Created
c	POST /comments (Content-Type Header)	201 Created
d	POST /comments (Content-Type + Authorization)	201 Created

F) In a real API, the Authorization header would contain a valid access token used to verify the user's identity. If the token were missing or invalid, the server would reject the request with an error such as **401 Unauthorized** or **403 Forbidden**, and the comment would not be created.

Submission Instructions

Submit your tasks inside your name folder in our GitHub repository. Make sure to include your requests and their response screenshots for each task.